Hack110 Sign-Up Form!

When: Saturday, April 18th from 10 AM - 12 AM (Midnight)

Where: Sitterson and Fred Brooks 1st floor (Floor 0)

Who can join? Anyone in COMP 110! No prior experience required.

Bring a partner, find one there, or go solo!

Come for a fun day of coding, workshops and events (**food and CLE credit will be provided**):

- Choose 1 of many tracks like web dev, game dev, etc - and WIN PRIZES too!
- Go to various **workshops & events** such as: Navigating the CS Major, Resume workshop, ice cream station, and kahoot trivia and MORE!
- Link: Sign-Up via the QR code
- **Sign-up form closes Tuesday, March 31st at 11:59 pm**
  - Spots are limited! So we'll prioritize interest!
  - If you have a partner, **ONLY ONE OF YOU** has to sign up - you will just enter your partner's info in the form.

Instagram

Sign-Up Here!

COMP 110

Unit Tests

Announcements

Quiz 02 this Thursday!

- Practice problems on the course site
  - Questions? Visit office hours/tutoring!
- Hybrid review session Wednesday evening (time and location TBA on the site today)
- Today’s material will only be covered conceptually on the quiz
  - Practice problems will be updated today with some relevant unit test questions

Write three functions

In your lessons folder, create a *folder* called unit_tests.

In unit_tests, create a file called list_fns.py

Write three functions:

- get_first: takes a list[str] as input and returns first element
- remove_first: takes a list[str] as input and removes first element (doesn't return anything)
- get_and_remove_first: takes a list[str] as input and returns + removes first element

Test them out to see if they work! :)

Unit Testing

- Writing functions to test other functions
  - Often multiple test functions for one function!
- Using pytest (other frameworks exist)

Syntax: Writing a unit test

Test file names: begin with name of module being tested, end with _test.py
Test function names: begin with test_

```
def test_name() -> None:
    assert <boolean expression>
```

Syntax: Writing a unit test

Test file names: begin with name of module being tested, end with _test.py
Test function names: begin with test_

```
def test_name() -> None:
    # Other code can go here!
    assert <boolean expression>
```

Syntax: Writing a unit test

Test file names: begin with name of module being tested, end with _test.py
Test function names: begin with test_

```
def test_name() -> None:
    # Other code can go here!
    assert <boolean expression>
```

Convention

Required by pytest

Two basic function behaviors you may want to test

- What it returns (Testing For Desired Output)
- How it modifies the input (Testing For Desired Behavior*)

Testing list_fns.py

Create a file in the same folder as list_fns.py called list_fns_test.py.

Begin the file by importing your list functions from list_fns.py:

```
from lessons.unit_tests.list_fns import get_first, remove_first, get_and_remove_first
```

Testing list_fns.py

Create a file in the same folder as list_fns.py called list_fns_test.py.

Begin the file by importing your list functions from list_fns.py:

```
from lessons.unit_tests.list_fns import get_first, remove_first, get_and_remove_first
```

Now we can write test functions!

Testing For Desired Output

- Checking that your function returns the correct output given a specific input

Example in VSCode…

Testing For Desired Behavior

- Checking that your function does what you want it to do rather than just checking what it returns.
- This can be useful for functions that *mutate* their input.

Example in VSCode…

Types of Tests

Use case: testing properties for how we expect our program to be used
Edge case: testing instances outside “typical” usage (e.g. empty inputs, incorrect inputs, etc.)

Both can test desired output or behavior!

Syntax: Running a unit test in the terminal

Command line: python -m pytest path/to/testfile.py

Alternatively: Use the Python Extension in VSCode's Testing Pane (the beaker icon in the leftmost sidebar)

Either way, ***make sure you are in the dev container!***